

实验一：语言设计与文法工程

一、实验概述

本实验是编译原理实验系列的第一个实验。整个学期的实验将围绕一个核心任务展开：每位同学需要自行设计一门编程语言，并在五个实验中逐步实现该语言的完整编译器（或解释器）。

本实验的重点是：（1）设计你自己的编程语言并撰写语言规范；（2）实现一个文法分析工具，能够读取任意上下文无关文法并进行 Chomsky 文法类型判定。

鼓励使用 Agent IDE（如 Claude Code、CodeBuddy、Cursor 等）辅助开发，但不做强制要求。无论是否使用 AI 工具，你都需要对自己的设计决策和代码有充分的理解，答辩时将重点考察这一点。

二、实验目标

- 理解 Chomsky 文法层次体系（0 型、1 型、2 型、3 型）的定义与判定方法
- 掌握正则文法（左线性/右线性）、上下文无关文法、上下文相关文法的形式化特征
- 能够独立设计一门编程语言的词法规范和语法规则
- 实现一个文法分析程序，对任意输入文法进行合法性验证和类型分类

三、实验内容

任务一：文法分析器（必做）

实现一个文法分析程序，能够读取标准格式的文法文件，对文法进行合法性验证和 Chomsky 类型判定。该程序将接入自动测评系统进行评分，因此必须严格遵守下述输入输出规范。

1. 输入格式

文法文件为纯文本文件（UTF-8 编码），格式如下：

```
# 非终结符集合（空格分隔）
N: S A B C
# 终结符集合（空格分隔）
T: a b c
# 产生式（每行一条，| 分隔同一左部的多个右部， $\epsilon$  表示空串）
P:
S -> A B C
A -> a A | a
B -> b
C -> c C |  $\epsilon$ 
# 开始符号
START: S
```

格式说明：

- 以 # 开头的行为注释，程序应跳过
- N: 后列出所有非终结符，空格分隔，建议用大写字母
- T: 后列出所有终结符，空格分隔
- P: 后每行一条产生式，左部和右部用 $\rightarrow$ 分隔，同一左部的多个候选式用 | 分隔
- ϵ （或 epsilon）表示空串
- START: 后为文法的开始符号，必须是 N 中的一个

2. 命令行接口

程序需支持以下命令行调用方式（可使用任意编程语言实现，但需提供可执行的启动方式）：

（1）文法类型判定

```
./grammar_analyzer input.txt --check-type
```

输出（仅一行，以下值之一）：

TYPE_3_RIGHT # 右线性正则文法
TYPE_3_LEFT # 左线性正则文法
TYPE_2 # 上下文无关文法（但不是正则文法）
TYPE_1 # 上下文相关文法
TYPE_0 # 无限制文法
INVALID # 文法不合法

判定规则：输出文法所属的最严格（最高编号）类型。例如，一个文法既符合 3 型也符合 2 型定义，应输出 TYPE_3_RIGHT 或 TYPE_3_LEFT。

(2) 文法合法性验证

./grammar_analyzer input.txt --validate

输出 JSON 格式：

合法时：
{ "valid": true }

不合法时：
{ "valid": false, "errors": [
 { "type": "UNDEFINED_SYMBOL", "symbol": "X", "location": "production 3" },
 { "type": "UNREACHABLE", "symbol": "D" },
 { "type": "NON_GENERATING", "symbol": "E" }
]}

需要检测的错误类型：

错误类型	含义	示例
UNDEFINED_SYMBOL	产生式中引用了未在 N 或 T 中定义的符号	N: S A, 但产生式出现 S -> A B
UNREACHABLE	从开始符号无法到达的非终结符	N 中有 D, 但无法从 S 推导出含 D 的句型
NON_GENERATING	无法推导出终结字符串的非终结符	A -> A B 且无其他 A 的产生式
START_NOT_IN_N	开始符号不在非终结符集合	START: X, 但 X 不在 N 中

	中	
FORMAT_ERROR	文件格式不符合规范	缺少 N:/T:/P:/START: 等必要部分

(3) 文法统计信息

```
./grammar_analyzer input.txt --stats
```

输出 JSON 格式:

```
{
  "num_nonterminals": 4,
  "num_terminals": 3,
  "num_productions": 6,
  "has_epsilon": true,
  "start_symbol": "S"
}
```

3. 文法类型判定规则

请按以下规则进行 Chomsky 层次分类:

3 型文法（正则文法）:

- 所有产生式形如 $A \rightarrow aB$ 或 $A \rightarrow a$ 或 $A \rightarrow \epsilon$ （右线性），或
- 所有产生式形如 $A \rightarrow Ba$ 或 $A \rightarrow a$ 或 $A \rightarrow \epsilon$ （左线性）
- 其中 A 、 B 为非终结符， a 为终结字符串（可为单个终结符或空）
- 注意：左线性和右线性不能混合，混合时归为 2 型

2 型文法（上下文无关文法）:

- 所有产生式左部为单个非终结符
- 右部可以是任意终结符和非终结符的组合

1 型文法（上下文相关文法）:

- 所有产生式满足 $|左部| \leq |右部|$ （左部长度不超过右部长度）
- 允许唯一例外： $S \rightarrow \epsilon$ ，但要求 S 不出现在任何产生式右部

0 型文法（无限制文法）:

- 不满足上述任何约束的文法

任务二：语言设计文档（必做）

本学期你将设计并实现一门自己的编程语言。本实验要求完成语言的顶层设计，产出语言规范文档。该文档将作为后续四个实验（词法分析、语法分析、语义分析、代码生成）的基础。

语言设计文档须包含以下内容：

（1）语言定位与设计哲学

- 你的语言面向什么场景或用户群体？
- 列出 3-5 条设计原则（例如：“简洁优先”、“强类型安全”等）
- 与已有语言的对比：你的语言在设计上与哪些语言相似？有哪些不同？

（2）词法规范

- 关键字列表
- 运算符和分隔符列表
- 标识符规则（用正则表达式定义）
- 各类字面量的格式（整数、浮点数、字符串等，用正则表达式定义）
- 注释格式

（3）语法规范

- 完整的 BNF 文法定义
- 运算符优先级和结合性表

（4）必考场景示例程序

使用你设计的语言，为下表中每个场景编写一段示例程序，并附简要说明：

编号	场景	考察点
S1	递归函数（如斐波那契、阶乘）	函数定义与调用、递归
S2	嵌套作用域与变量遮蔽	作用域规则、符号表

S3	类型检查与类型转换	类型系统
S4	控制流嵌套（if/循环多层嵌套）	控制流
S5	数组或复合数据结构操作	内存布局、索引
S6	错误程序（词法/语法/语义错误各一）	错误检测与报告

注意：这六个场景将贯穿整个学期。后续每个实验都需要验证你的编译器能正确处理这些场景。请认真设计，因为语言规范一旦确定，后续修改的成本会越来越高。

任务三：二义性检测（选做，加分项）

在文法分析器中增加二义性检测功能。

命令行接口：

```
./grammar_analyzer input.txt --ambiguity-check --max-length 8
```

输出：

```
# 发现二义性时：
{"ambiguous": true, "witness": "a b c", "parse_count": 2}

# 未发现二义性时：
{"ambiguous": false, "max_length_checked": 8}
```

实现思路：对给定文法，穷举长度不超过 max-length 的所有终结字符串，尝试寻找存在两棵不同语法分析树的字符串。如果找到，输出该字符串作为反例。

四、交付物清单

序号	交付物	文件/格式	说明
1	文法分析器源代码	源代码文件	可使用任意编程语言
2	构建/运行说明	README.md	说明如何编译和运行你的程序

3	语言设计文档	language_spec.pdf 或 .docx	≥3000 字
4	语言的 BNF 文法文件	grammar.ebnf	纯文本，后续实验使用
5	必考场景示例程序	scenarios/S1.src ~ S6.src	6 个文件，使用你的语言编写
6	测试文法集	tests/	你自己编写的测试文法文件（≥10 个，覆盖四种类型）
7	设计决策记录	decisions.md	记录关键设计决策及其理由

五、考核方式

项目	占比	评分方式	说明
文法分析器正确性	50%	自动测评	使用标准测试集，对比程序输出与标准答案
语言设计文档	30%	实验报告	重点考察设计决策的合理性和对文法理论的理解
代码质量与自测	10%	自动+人工	代码可读性、自测文法的覆盖度
二义性检测（选做）	10%（加分）	自动测评	使用含/不含二义性的文法测试

六、实验报告要点

重点阐述以下问题，请提前准备：

- Chomsky 四型文法的定义与判定方法，请举例说明各型文法的区别
- 你的文法分析器的判定算法是如何实现的？时间复杂度如何？
- 你设计的语言有什么特点？为什么这样设计？有哪些备选方案被你否决了？
- 如果让你重新设计，你会做哪些改变？

七、自动测评说明

本实验将使用自动测评系统对文法分析器进行评分。测评系统会准备一批文法文件，覆盖以下情况：

- 标准 3 型文法（左线性和右线性各若干）
- 标准 2 型文法（含递归产生式、 ϵ 产生式、单产生式等）
- 1 型文法（上下文相关的典型用例）
- 0 型文法
- 非法文法（格式错误、引用未定义符号、不可达非终结符等）
- 边界用例（空产生式链、循环产生式、单产生式等）

测评流程：

- 对每个测试文法，依次调用 `--check-type`、`--validate`、`--stats`
- 将你的程序输出与标准答案逐项比对
- 每个测试用例独立计分，最终取加权平均

重要提示：

- 请确保你的程序在 Linux 环境下可运行（测评服务器为 Linux）
- 程序必须在 10 秒内完成单个文法文件的分析，超时视为失败
- 输出必须严格遵守上述格式，多余的空格、换行或调试信息都可能导致判定失败
- 建议在 `stdout` 输出结果，`stderr` 输出调试信息（测评系统只读取 `stdout`）

八、参考资料

- Chomsky, N. (1956). Three models for the description of language.
- Aho, Lam, Sethi, Ullman. Compilers: Principles, Techniques, and Tools (Dragon Book), Chapter 2, 4
- Grune, Jacobs. Parsing Techniques: A Practical Guide, Chapter 2
- EBNF 标准: ISO/IEC 14977